CPS209 Lab 4

Expanding and Extending Polynomial

Preamble

In this lab you will build upon the Polynomial class you wrote in lab 2. You will add some new methods, as well as create a brand-new child class that extends Polynomial.

Homework to Complete (Submit on D2L):

This lab requires you to do work in two Java files: Polynomial.java (the same one from lab 3) and BigO.java. BigO.java is not given – you must create it yourself from scratch. You will define the class, create your own constructor, add your own methods, etc. An updated tester for Polynomial.java is given, as well as a separate tester for BigO.java. The method definitions MUST follow the definitions given below, or they won't work with the unit tests.

Reminders:

- Your methods should be SILENT. Print statements are fine for testing and debugging but remove them from your code before you submit.
- No method should take longer than a couple of seconds to pass the tester. If your code takes minutes (or *hours*, heaven help you) it means you are doing something very inefficient in your approach and/or implementation. Remedy this before submitting. Our TAs will be very busy and cannot wait 20 minutes for a single student's code to execute.

Method Descriptions – Polynomial.java:

Your Polynomial class should still contain the following methods from Lab #3:

```
public Polynomial(int[] coefficients)
public int getDegree()
public int getCoefficient(int k)
```

You will add three new methods to the Polynomial class, described below:

`public long evaluate(int x)`

This method evaluates the polynomial using the value x for the unknown symbolic variable of the polynomial. For example, when called with x=2 for the polynomial with coefficients {42,-7,0,5}, this method should return 68. This method should avoid floating-point operations at all costs! This means you should NOT just naively call **`Math.pow()`**, as this accepts and returns double and will implicitly cast your integer arguments as doubles in the process.

`public Polynomial add(Polynomial other)`

This method creates and returns a new Polynomial object that represents the result of polynomial addition of the two polynomials this and other. Make sure that the coefficient of the highest term of the result is nonzero, so that adding two polynomials $5x^{10} - x^2 + 3x$ and $-5x^{10} + 7$, both having the same degree of 10, produces the result $-x^2 + 3x + 7$ that has a degree of only 2. Note that neither the "this" Polynomial or other should be changed by this operation. Instead, an entirely new object must be created and returned.

`@Override`
`public String toString()`

This method will return a String representation of the polynomial. The String representation of the polynomial should only include terms whose coefficient is non-zero. Several examples can be seen below. The unit tests will show you other examples. Notice that there is a space character in between the terms and the symbolic operators, though there should be no space before the first term if it is negative (see the third example below). In addition, there should be no exponent for the linear term (ie. it should be **7x** not **7x^1**), and the constant term should not show x at all (ie. it should be **42** instead of **42x^0**).

`{42,-7,0,5}` **`"5x^3 - 7x + 42"`**
`{0,0,0,0,1}` **`"1x^4"`**
`{1,2,-3,0,-7}` **`"-7x^4 - 3x^2 + 2x + 1"`**

Method Descriptions – BigO.java:

This class represents the Big-O time complexity of a polynomial and must extend the Polynomial class. The Big-O representation of a polynomial is simply the highest order term, with the coefficient chopped off. For example:

$5n^{10} + n^2 + 3n = O(n^{10})$
$n^2 + 3n + 7 = O(n^2)$

Implement the following constructor and methods in BigO.Java:

`public BigO(int[] coefficients)`

This constructor should invoke the superclass constructor in Polynomial, passing the coefficients as an argument. Additionally, this constructor should initialize a private instance variable of type String, called "category". The value of category will depend on the degree of the polynomial:

Degree 0 "constant"

Degree 1	“linear”
Degree 2	“quadratic”
Degree 3	“cubic”
All others	“polynomial”

`public String getCategory ()`

This method simply returns the category.

`@Override`
`public String toString()`

The String representation of a BigO object is based solely on the degree. If the degree is 0, return **`“O(1)”`**. If the degree is 1, return **`“O(n)”`**. In general, if the degree is **k**, then the string representation should be returned as **`“O(n^k)”`**.

Notes and Hints:

Don’t forget that BigO inherits all public methods from Polynomial, such as **`getDegree()`**, **`getCoefficients()`**, etc. You can call these in your BigO class. Additionally, you are not limited to the methods and variables described above. You may include any additional helper methods and instance variables that you find useful.

Finally, we suggest you start creating your BigO class by adding “method stubs” for all required methods like we did in the original Polynomial.java file we gave you. These consist of the method signature, and a “dummy” return value (for example, “getCategory” can be set to initially return the empty string). The objective in doing so is to allow the code to compile, so that you can run the unit tests. This will let you focus on implementing one method at a time, after which you can test the method you are working on, before moving on to the next method. This usually makes it easier to handle bugs, as you won’t have to deal with multiple issues at the same time.

Submission

Labs are to be completed and submitted ***individually***. Submit your completed **BigO.java** and **Polynomial.java** on D2L, under the submission for Lab 4.